

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
import urllib.request

# 1. Download the Sample Image (with error handling and synthetic fallback)
# Corrected up-to-date URL pointing to OpenCV's current active '4.x' branch
image_url = "https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/moon.jpg"

try:
    print("Attempting to download image...")
    # Add a User-Agent header to prevent servers from blocking python's default crawler agent
    req = urllib.request.Request(image_url, headers={'User-Agent': 'Mozilla/5.0'})
    with urllib.request.urlopen(req) as response:
        with open("moon.jpg", "wb") as f:
            f.write(response.read())

    # Load image in grayscale and normalize to [0, 1]
    clean_scene = cv2.imread("moon.jpg", cv2.IMREAD_GRAYSCALE)
    if clean_scene is None:
        raise ValueError("Image file corrupted during download.")
    print("Download successful!")
except Exception as e:
    print(f"Download failed ({e}). Generating a synthetic moon-like scene mathematically as a fail-safe...")
    # Create a 512x512 synthetic moon (a gradient sphere with craters)
```

```

y, x = np.ogrid[-256:256, -256:256]
mask = x**2 + y**2 <= 180**2
# Create moon texture using radial gradients and random crater-like depressions
clean_scene = np.zeros((512, 512))
r = np.sqrt(x**2 + y**2)
clean_scene[mask] = 0.8 * (1 - (r[mask]/180)**2)**0.3
# Add simulated craters
np.random.seed(42)
for _ in range(15):
    cx, cy = np.random.randint(-120, 120, size=2)
    cr = np.random.randint(10, 35)
    crater_mask = (x - cx)**2 + (y - cy)**2 <= cr**2
    clean_scene[crater_mask & mask] *= 0.7

clean_scene = cv2.resize(clean_scene, (512, 512))
# Ensure values are normalized floats [0, 1]
if clean_scene.max() > 1.0:
    clean_scene = clean_scene.astype(float) / 255.0

# 2. Simulate Low-Light Acquisition Pipeline
def capture_low_light_frame(scene, photon_level=5.0, read_noise_std=0.05, iso_gain=2.0):
    """
    Simulates the physical low-light capture process.
    - photon_level: Determines the maximum expected number of photons hitting a pixel (Poisson Mean).
    - read_noise_std: Standard deviation of electronic read noise (Gaussian distribution).
    - iso_gain: Amplification multiplier.
    """
    # Step A: Photon Shot Noise (Poisson statistics)
    photon_counts = np.random.poisson(scene * photon_level).astype(float)

    # Step B: Electronic Read Noise (Gaussian noise floor added by sensor electronics)
    electronic_noise = np.random.normal(0, read_noise_std, scene.shape)
    raw_signal = photon_counts + electronic_noise

    # Step C: ISO Amplification
    amplified_signal = raw_signal * iso_gain

    # Clip values to standard image boundaries [0, 255]
    final_frame = np.clip(amplified_signal * (255.0 / (photon_level * iso_gain)), 0, 255).astype(np.uint8)
    return final_frame

# Set parameters for a dark low-light environment
photon_level = 3.0 # Low number of photons reaching the sensor
read_noise_std = 0.8 # High electronic read noise floor
iso_gain = 4.0 # High ISO amplification to make the image visible

# Simulate capturing a single noisy low-light frame
single_frame = capture_low_light_frame(clean_scene, photon_level, read_noise_std, iso_gain)

# 3. Software Correction: Multi-Frame Temporal Fusion
num_frames = 16
burst_frames = [capture_low_light_frame(clean_scene, photon_level, read_noise_std, iso_gain).astype(float) for _ in range(num_frames)]

# Mathematically average the frames to reduce random noise
fused_frame = np.mean(burst_frames, axis=0).astype(np.uint8)

# 4. Plot and Compare the Results
plt.figure(figsize=(20, 8))

# Plot Original Ground Truth
plt.subplot(1, 3, 1)
plt.imshow(clean_scene, cmap='gray')
plt.title("1. Original Ideal Scene\n(Perfect Illumination)", fontsize=12)
plt.axis('off')

# Plot Raw Low-Light Capture (One Frame)
plt.subplot(1, 3, 2)
plt.imshow(single_frame, cmap='gray')
plt.title(f"2. Single Low-Light Frame\n(Shot + Read Noise amplified by ISO)", fontsize=12)
plt.axis('off')

# Plot Restored Image (Multi-Frame Fusion)
plt.subplot(1, 3, 3)
plt.imshow(fused_frame, cmap='gray')
plt.title(f"3. Multi-Frame Fusion ({num_frames} frames)\n(Temporal Averaging Recovery)", fontsize=12)
plt.axis('off')

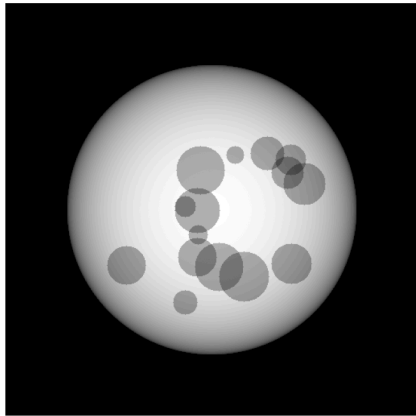
plt.tight_layout()
plt.show()

```

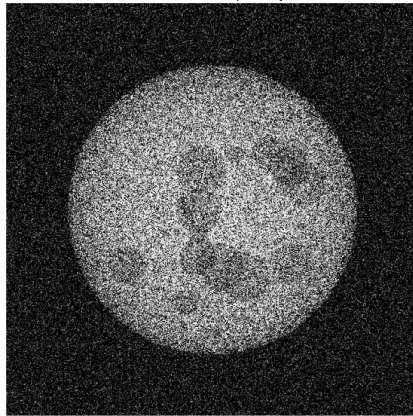
Attempting to download image...

Download failed (HTTP Error 404: Not Found). Generating a synthetic moon-like scene mathematically as a fail-safe...

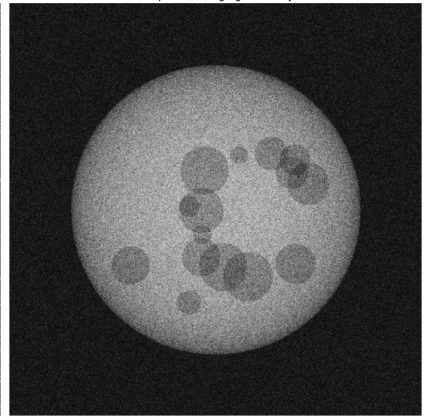
1. Original Ideal Scene
(Perfect Illumination)



2. Single Low-Light Frame
(Shot + Read Noise amplified by ISO)



3. Multi-Frame Fusion (16 frames)
(Temporal Averaging Recovery)



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs](#) and [TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

More Resources

Working with Notebooks in Colab

- [Overview of Colab](#)
- [Guide to Markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with Data

- [Loading data: Drive, Sheets, and Google Cloud Storage](#)
- [Charts: visualizing data](#)
- [Getting started with BigQuery](#)

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Getting Started with cuML's accelerator mode](#)

Using Accelerated Hardware

- [Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API](#)
- [Train a Vision Transformer \(ViT\) for image classification with JAX](#)
- [Text classification with a transformer language model using JAX](#)

Featured examples

- [Train a miniGPT language model with JAX AI Stack](#)
- [LoRA/QLoRA finetuning for LLM using Tunix](#)
- [Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA](#)
- [Loading Hugging Face Transformers Checkpoints](#)
- [8-bit Integer Quantization in Keras](#)
- [Float8 training and inference with a simple Transformer model](#)
- [Pretraining a Transformer from scratch with KerasHub](#)
- [Simple MNIST convnet](#)
- [Image classification from scratch using Keras 3](#)
- [Image Classification with KerasHub](#)